SOLID in Practice

7 min read Updated 28 Jul 2026 Topic 63 of 65

Bookmark Mark done

Why it matters

SOLID is the most-recited and least-understood acronym in software. Anyone can expand the letters; what interviewers listen for is whether you can point at code and say *which* principle it breaks and what the concrete cost is. Recited definitions are worth nothing; "this class has two reasons to change, and that is why the deployment broke when we altered the email template" is worth a great deal.

The concept, simply

	Principle	The practical test
S	Single responsibility	How many different reasons would make you edit this class?
O	Open/closed	Can you add a case without editing existing code?
L	Liskov substitution	Can a subclass replace its parent without surprising callers?
I	Interface segregation	Are implementers forced to stub methods they do not need?
D	Dependency inversion	Does the policy depend on an abstraction or on a detail?

How it works

Single responsibility is not "a class does one thing" — that is too vague to apply. It is "a class has one *reason to change*", one stakeholder. A class that formats a report and also

JAVA Copy

```java
// ✗ Three reasons to change: business rules, persistence, and messaging.
class OrderService {
    void place(Order o) {
        if (o.total().signum() ≤ 0) throw new IllegalArgumentException();
        jdbc.update("INSERT INTO orders …");
        mailer.send(o.customerEmail(), "Order confirmed", buildHtml(o));
    }
}

// ✓ Each collaborator changes for its own reason; the service coordinates
class OrderService {
    private final OrderRepository repository;
    private final OrderNotifier notifier;
    void place(Order o) { o.validate(); repository.save(o); notifier.confi
}
```

Open/closed in modern Java usually means replacing a growing `switch` over a type code with polymorphism — or, since Java 21, with a sealed hierarchy where the compiler *forces* you to handle each case (Chapter 10).

JAVA Copy

```java
// ✗ Every new payment method edits this method — and every other switch
//   over the same enum elsewhere, which you have to go and find.
BigDecimal fee(Payment p) {
    switch (p.type()) {
        case CARD: return p.amount().multiply(new BigDecimal("0.029"));
        case UPI:  return BigDecimal.ZERO;
        default:   throw new IllegalArgumentException();   // ✗ runtime fa
    }
}

// ✓ Polymorphic: a new method is a new class, and nothing existing change
sealed interface Payment permits Card, Upi, BankTransfer {
    BigDecimal fee();
}
```

```
// ✓ Better still, a sealed hierarchy with an exhaustive switch — adding a
//   COMPILE error in every switch that does not handle it.
//   Extensible AND checked, which classic polymorphism cannot give you.
```

Liskov is the one with real teeth, and Java's own library breaks it:

JAVA Copy

```
List<String> list = Arrays.asList("a", "b");
list.add("c");        // ✗ UnsupportedOperationException at run time

// It IS a List by type and does not honour List's contract. Any method
// written against List can be handed this and fail. Chapter 2's override
// rules — covariant returns, no narrowing of access, no broader checked
// exceptions — are the language enforcing Liskov where a compiler can.
// Weakening a postcondition at run time is the part it cannot catch.
```

Interface segregation and **dependency inversion** are the two Spring makes almost automatic — a fat interface hurts the moment you try to mock it, and constructor injection against an interface *is* dependency inversion.

JAVA Copy

```
// ✗ Fat interface: an in-memory implementation must stub half of it.
interface Repository { void save(); void delete(); void export(); void bac

// ✓ Segregated: implementers take only what they can honour.
interface OrderReader { Optional<Order> findById(String id); }
interface OrderWriter { void save(Order o); }

// ✓ Dependency inversion: the policy depends on an abstraction, and so do
//   the implementation — both point at the middle rather than one at the
class OrderService {
    private final OrderRepository repository;      // interface
    OrderService(OrderRepository repository) { this.repository = repositor
}
```

Under the hood

SOLID is a set of heuristics, not laws — and saying so is a better answer than reciting them, because over-application is a real and common failure. An interface for every class with one implementation, a factory for every construction, four layers of indirection to change a string: that is not clean, it is *speculative generality*, and it trades real readability for flexibility nobody asked for. The time to introduce an abstraction is when you have a second case or a genuine testing seam — not in anticipation of one.

Modern Java changes how you satisfy them. Sealed interfaces plus exhaustive switches give open/closed *with* compile-time checking. Records give immutable value objects with no boilerplate, removing much of the state that single-responsibility violations accumulate around. Constructor injection makes dependency inversion the path of least resistance rather than an extra effort.

The honest framing: every principle is a different angle on one question — when this requirement changes, how much has to change with it? A design that scores well on the acronym but makes the common change hard has missed the point.

Common mistakes

- **Reciting definitions without examples.** The example is the answer.
- **Reading SRP as "one method".** It is one reason to change.
- **An interface per class by reflex.** One implementation and no test seam is pure indirection.
- **Inheriting to reuse code.** Inheritance is an "is-a" claim; use composition (Chapter 2).
- **A `switch` that grows forever.** Polymorphism or a sealed hierarchy.
- **Throwing `UnsupportedOperationException` from an override.** A Liskov violation by definition.
- **Applying all five up front.** Speculative generality has its own cost.

Interview lens

> **Q: "Explain SOLID with real examples."**
> A: Single responsibility — one reason to change; a service that also builds SQL and formats email changes for three different stakeholders. Open/closed — a growing

parent's contract, and `Arrays.asList` returning a `List` that throws on `add` breaks it. Interface segregation — fat interfaces force implementers to stub methods. Dependency inversion — depend on an interface, which constructor injection makes natural.

Q: "Give an example of a Liskov violation."
A: The JDK has a good one: `Arrays.asList` returns something typed `List` whose `add` throws `UnsupportedOperationException`, so any method written against `List` can be handed it and fail. The teaching example is `Square extends Rectangle` — setting the width changes the height, so code reasoning about a rectangle gets a surprise. Java enforces what it can through the override rules, but it cannot stop you weakening a postcondition.

Q: "Is SOLID always right?"
A: They are heuristics for making change cheap, not laws. Over-applying them produces speculative generality — an interface for every class, a factory for every constructor, indirection that makes code harder to read for flexibility nobody needed. I add an abstraction when there is a second implementation or a real testing seam, not before.

Q: "How does modern Java change this?"
A: Sealed interfaces with exhaustive switches give open/closed plus compile-time checking, which plain polymorphism cannot — adding a case breaks the build in every switch that ignores it. Records remove the boilerplate around immutable value objects, which is where a lot of single-responsibility drift begins. And constructor injection makes dependency inversion the easiest thing to write rather than an extra step.

Cheat-card

- S — one *reason to change*, not one method.
- O — add a case without editing existing code → polymorphism or **sealed + exhaustive switch**.
- L — honour the contract; `Arrays.asList().add()` is the JDK's own violation.
- I — no forced stub methods; small interfaces.
- D — depend on abstractions; constructor injection gives it for free.
- Java's **override rules enforce Liskov** where a compiler can.
- They are **heuristics** — over-application is speculative generality.
- The question behind all five: **when this changes, how much changes with it?**

QUICK CHECK — no marks, just for you.

1. What does the Single Responsibility Principle actually say?

- ◯ A method should do only one thing
- ◯ A class should have only one field
- ◯ A class should have only one public method
- ◯ A class should have one REASON TO CHANGE — one stakeholder — not merely 'do one thing'

Check answer

2. Which is a Liskov substitution violation in the JDK itself?

- ◯ Arrays.asList returning something typed List whose add() throws UnsupportedOperationException
- ◯ String being final
- ◯ ArrayList implementing both List and RandomAccess

Check answer

3. What is the risk of applying SOLID too aggressively?

- ◯ Larger class files
- ◯ Speculative generality — an interface for every class, a factory for every constructor, layers of indirection that cost readability for flexibility nobody needed
- ◯ There is no risk; more abstraction is always better
- ◯ Slower run-time performance

Check answer

‹ Previous
Testing: JUnit 5, Mockito and Spring Slices

Next ›
Design Patterns That Still Earn Their Keep

LEARN MORE

ARTICLES & DOCS

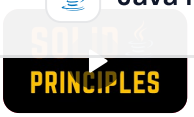
- Liskov substitution and subtyping (JLS §8.4.8)
  docs.oracle.com
- Interfaces — The Java Tutorials
  docs.oracle.com

VIDEOS



SOLID Design Principles in java with Example | JavaTechie
Java Techie

Java Interview Kit



SOLID Design Principles with Java Examples | Clean Code and Best Practices | Geekific

Geekific

Hand-picked references for this lesson.